Lab Assignment-2

School of Engineering and Technology
Course Name & Code: Operating System Lab ENCA252
Program Name: BCA (AI & DS) (Research)

Submission Guidelines

General Requirements

- **Submission Deadline:** Assignments must be submitted within one week of the assignment's release date.
- **Submission Platform:** All assignments are to be submitted via the Learning Management System (LMS).
- **GitHub Link:** You must provide a link to your GitHub repository with your submission.
- **Individual Submission:** Assignments are to be completed and submitted by each individual student.
- **Formatting:** All assignments must adhere to the specific format shared in class.

Evaluation

- **Total Marks:** This assignment is worth a total of 5 marks.
- **Evaluation Metrics:** Assignments will be evaluated based on the following criteria:
    - **Originality:** The uniqueness and independent thought demonstrated in the work.
    - **Correctness:** The accuracy and validity of the solutions or content.

Problem Title:

Implementation of Banker's Algorithm for Deadlock Avoidance

Problem Statement

Deadlock is a situation in operating systems where a set of processes are unable to proceed because each is waiting for resources held by others. Banker's Algorithm is a deadlock avoidance technique that ensures the system remains in a safe state. In this assignment, students will implement Banker's Algorithm and determine whether the system is in a safe state, as well as find the safe sequence of execution.

Learning Objectives

• Understand CPU scheduling concepts
• Implement FCFS and SJF algorithms
• Calculate CT, TAT, WT
• Analyze system performance

Tools/Technology Used:

• Python 3.x
• Built-in Python libraries: os, multiprocessing, time
• Linux OS (optional for simulation)

Assignment Tasks:

Task 1: System Input and Data Representation

Objective:

To understand how resources are allocated to processes in an operating system.

Description:

Students will input system data including number of processes, number of resources, allocation matrix, maximum matrix, and available resources.

Sub-Tasks:

1. Define number of processes and resources.
2. Input Allocation Matrix.
3. Input Maximum Matrix.
4. Input Available Resources.

Expected Outcome:

· Understanding of system resource allocation
· Ability to represent system state

Task 2: Need Matrix Calculation

Objective:

To compute the remaining resource requirements for each process.

Description:

Need Matrix is calculated using the formula:
Need = Maximum – Allocation

Sub-Tasks:

1. Calculate Need Matrix.
2. Display the Need Matrix.

Expected Outcome:

· Clear understanding of process requirements
· Ability to derive required resources

Task 3: Banker's Safety Algorithm Implementation

Objective:

To determine whether the system is in a safe state.

Description:

The safety algorithm checks if there exists a safe sequence in which all processes can execute without causing deadlock.

Sub-Tasks:

1. Initialize Work = Available.
2. Initialize Finish array for all processes.
3. Find a process whose Need ≤ Work.
4. Allocate resources and update Work.
5. Repeat until all processes are completed or no further allocation is possible.

Expected Outcome:

· Ability to check safe/unsafe state
· Understanding of deadlock avoidance

Task 4: Safe Sequence Determination

Objective:

To find the safe execution order of processes.

Description:

If the system is in a safe state, a sequence of execution is generated.

Sub-Tasks:

1. Store processes in safe sequence.
2. Display the safe sequence.

Expected Outcome:

· Understanding of execution order
· Visualization of safe process flow

Task 5: Result Analysis

Objective:

To analyze system safety and interpret results.

Description:

Students will evaluate whether the system is safe or unsafe and justify their results.

Sub-Tasks:

1. Identify whether the system is in safe or unsafe state.
2. Explain the result.
3. Compare with theoretical expectations.

Expected Outcome:

· Analytical thinking
· Understanding of real-world deadlock scenario

Student Instructions:

Submit all Python source code files and screenshots of output.
Maintain proper code structure and add meaningful comments.
Submit a summary report in .pdf or .docx format explaining your code and outputs.
Late submissions will lead to grade penalties as per academic policy.

Rubrics for Evaluation

Criterion	**Weight**	**Excellent (5)**	**Good (4)**	**Fair (3) or Poor (2 or less)**
Implementation & Execution	40%	Correct simulation of all components	Minor logic or output issues	Significant errors or missing outputs
Code Quality & Documentation	30%	Well-commented and structured code	Mostly readable, few comments	Poor formatting or no documentation
Output Accuracy	20%	All outputs correct and validated	Minor output mismatches	Incomplete or incorrect output
Report & Submission	10%	Submitted on time, clear report	Minor issues or delay	Late or no report